# Hilt + Dependency Injection

Deep dive · Hilt internals · code generation · scopes · testing · KMP

2026 Edition | Original prose | Side-by-side bytecode | Interview-ready

**About this sheet:** a dense reference on Dependency Injection in Android with depth on Hilt — including what its compiler actually generates. Same UI shell as the other cheat sheets in this repo. Original prose, standard idiomatic patterns, interview-flavored.

## 1. Why DI — what problem it solves

Without DI, every class builds the things it depends on. Looks fine for one class. Becomes unmaintainable across an app:

```
// Without DI
class UserViewModel {
    private val retrofit = Retrofit.Builder()...build()
    private val api = retrofit.create(UserApi::class.java)
    private val db = Room.databaseBuilder(...).build()
    private val repo = UserRepository(api, db.userDao())
    suspend fun load() = repo.getUser()
}
// Problems:
// 1. Each ViewModel makes its own Retrofit/DB → wasted resources, duplicate config
// 2. Cannot swap real Repo for FakeRepo in tests
// 3. Changing Retrofit setup means editing every consumer
// 4. Hidden dependencies — class signature lies about what it really needs
```

DI flips the responsibility: classes **declare** what they need; somebody else **provides** it.

```
// With DI
class UserViewModel(private val repo: UserRepository) : ViewModel() {
    suspend fun load() = repo.getUser()
}
// Now: testable (pass a FakeRepository), reusable, clear contract.
// "Somebody else" = the DI framework — Hilt, Koin, or manual.
```

### What DI gives you

- **Testability**: substitute fakes/mocks at test time
- **Single source of truth**: one Retrofit, one DB, shared everywhere

- **Explicit contracts**: constructor lists everything the class needs
- **Decoupling**: Repo doesn't know which DAO/API impl was injected
- **Lifecycle management**: framework knows when to clean up scope-bound objects

## 2. Manual DI vs framework DI — when each fits

| | MANUAL DI | FRAMEWORK DI |
|---|---|---|
| Setup cost | Zero — just plain Kotlin classes | Some Gradle config + learning curve |
| Boilerplate at scale | Grows linearly with deps; painful | Constant — framework wires automatically |
| Scope lifecycle | You manage it | Framework gives you @Singleton, @ViewModelScoped, etc. |
| Compile-time validation | You spot bugs at runtime | Hilt/Dagger catch missing bindings at compile |
| Build-time impact | None | KSP/kapt processing time |
| Best for | Single-screen apps, small libraries | Real production apps |

### Manual DI example — a Service Locator container

```
object AppContainer {
    lateinit var applicationContext: Context

    val okHttpClient: OkHttpClient by lazy { OkHttpClient.Builder().build() }
    val retrofit: Retrofit by lazy {
        Retrofit.Builder().client(okHttpClient).baseUrl("...").build()
    }
    val userApi: UserApi by lazy { retrofit.create(UserApi::class.java) }
    val database: AppDatabase by lazy {
        Room.databaseBuilder(applicationContext, AppDatabase::class.java, "app").build()
    }
    val userRepository: UserRepository by lazy {
        UserRepositoryImpl(userApi, database.userDao())
    }
}

// In Application
override fun onCreate() {
    super.onCreate()
    AppContainer.applicationContext = this
}

// In a ViewModel
class UserViewModel : ViewModel() {
    private val repo = AppContainer.userRepository
}
```

Works for small apps. At ~20+ classes you start fighting it (every consumer needs to know `AppContainer` exists; testing requires reassigning singletons; child scopes don't exist). Time to pick a framework.

## 3. Hilt vs Koin vs Dagger — when to pick which

| | HILT | KOIN | DAGGER |
|---|---|---|---|
| What it is | Dagger preconfigured for Android (built on top) | Pure Kotlin DSL, runtime resolution | The original — JSR-330 annotation processor |
| Validation | Compile-time (KSP/kapt) | Runtime (Koin Verify catches at test time) | Compile-time |
| Boilerplate | Low (Android sugar) | Lowest (DSL) | High (manual components) |
| Build cost | KSP/kapt processing time | None | kapt processing time |
| Runtime cost | Negligible (codegen) | Reflection on first init, then cached | Negligible (codegen) |
| Android scopes built-in | Yes (Singleton, ActivityRetained, ViewModel, Activity, Fragment, View, Service) | Manual (you define scope hierarchy) | Manual (you define every component) |
| KMP-friendly | No (Android-only) | Yes (multiplatform) | Limited |
| IDE support | Excellent (Android Studio specific) | OK | OK |
| Best for | Most Android apps | KMP shared modules, smaller apps | Multi-module non-Android JVM, custom DI |

> **2026 default:** Hilt for Android. Koin if you're sharing a module across Android + iOS via KMP. Manual DI for libraries you publish (don't force a framework on consumers).

## 4. Hilt setup — minimal working app

### Step 1 — Gradle

```
// project/build.gradle.kts
plugins {
    id("com.google.dagger.hilt.android") version "2.51.1" apply false
    id("com.google.devtools.ksp") version "2.0.0-1.0.21" apply false
}

// app/build.gradle.kts
plugins {
    id("com.android.application")
    kotlin("android")
    id("com.google.dagger.hilt.android")
    id("com.google.devtools.ksp")
}

dependencies {
    implementation("com.google.dagger:hilt-android:2.51.1")
    ksp("com.google.dagger:hilt-android-compiler:2.51.1")
    implementation("androidx.hilt:hilt-navigation-compose:1.2.0")
}
```

## Step 2 — Application class

```
@HiltAndroidApp
class App : Application()
```

Set `android:name=".App"` in the manifest. Hilt's Gradle plugin actually rewrites it to a generated subclass (`Hilt_App`) — you don't see it in your code.

## Step 3 — Mark Android entry points

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    @Inject lateinit var analytics: Analytics
    ...
}

@HiltViewModel
class UserViewModel @Inject constructor(
    private val repo: UserRepository
) : ViewModel()
```

## Step 4 — Provide things Hilt can't construct itself

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides @Singleton
    fun provideOkHttp(): OkHttpClient = OkHttpClient.Builder().build()

    @Provides @Singleton
    fun provideRetrofit(client: OkHttpClient): Retrofit = Retrofit.Builder()
        .baseUrl("https://api.example.com/")
        .client(client)
        .build()

    @Provides @Singleton
    fun provideUserApi(retrofit: Retrofit): UserApi = retrofit.create()
}
```

That's it. Now `UserViewModel` works — Hilt sees it needs a `UserRepository`, finds `UserRepositoryImpl` via the binding (next sections), which needs `UserApi` + `UserDao`, both of which it knows how to provide. Wires everything automatically.

## 5. Components and scopes — the full hierarchy

A **component** is a container of bindings tied to an Android lifecycle. A **scope** is an annotation that marks a binding to live as long as its owning component. Hilt ships predefined components — you almost never define custom ones.

```
┌──────────────────────────────────────────────────────────────────────┐
│  SingletonComponent (lives for app process)                          │
│  Scope: @Singleton                                                   │
│  Use: Database, OkHttpClient, Retrofit, app-wide repos               │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  ActivityRetainedComponent (survives config change)          │   │
│   │  Scope: @ActivityRetainedScoped                              │   │
│   │  Use: Shared state across Fragments / config-survive          │   │
│   │                                                              │   │
│   │   ┌──────────────────────────────────────────────────────┐   │   │
│   │   │  ViewModelComponent (per ViewModel instance)         │   │   │
│   │   │  Scope: @ViewModelScoped                             │   │   │
│   │   │  Use: Per-VM helpers                                 │   │   │
│   │   └──────────────────────────────────────────────────────┘   │   │
│   │                                                              │   │
│   │   ┌──────────────────────────────────────────────────────┐   │   │
│   │   │  ActivityComponent (re-created on rotation)          │   │   │
│   │   │  Scope: @ActivityScoped                              │   │   │
│   │   │                                                      │   │   │
│   │   │   ┌──────────────────────────────────────────────┐   │   │   │
│   │   │   │  FragmentComponent                           │   │   │   │
│   │   │   │  Scope: @FragmentScoped                      │   │   │   │
│   │   │   │                                              │   │   │   │
│   │   │   │   ┌──────────────────────────────────────┐   │   │   │   │
│   │   │   │   │  ViewComponent / ViewWithFragment    │   │   │   │   │
│   │   │   │   │  Scope: @ViewScoped                  │   │   │   │   │
│   │   │   │   └──────────────────────────────────────┘   │   │   │   │
│   │   │   └──────────────────────────────────────────────┘   │   │   │
│   │   └──────────────────────────────────────────────────────┘   │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  ServiceComponent                                            │   │
│   │  Scope: @ServiceScoped                                       │   │
│   └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

## What each scope means in plain English

| SCOPE | LIVES FOR | TYPICAL USE |
|---|---|---|
| @Singleton | Entire app process | OkHttpClient, AppDatabase, Retrofit, top-level Repository |
| @ActivityRetainedScoped | Survives configuration change; cleared when Activity finishes for real | Multi-Fragment shared state, per-flow controller state (checkout flow VMs use this) |
| @ViewModelScoped | Per ViewModel instance | Per-VM helpers (rare; usually constructor-injected) |
| @ActivityScoped | Per Activity instance (re-created on rotation) | Per-Activity caches |
| @FragmentScoped | Per Fragment instance | Per-Fragment helpers |
| @ViewScoped | Per View instance | Custom View helpers |
| @ServiceScoped | Per Service instance | Service-level helpers |

### Rule of thumb — pick the smallest scope that satisfies the need

- **Don't @Singleton everything.** Singletons survive forever; if they hold any state, that state survives forever too.
- **Stateless services** (mappers, utilities) → no scope (Hilt creates a fresh instance on each request — usually negligible cost)
- **Heavy stateful objects** (Database, OkHttp pool) → @Singleton
- **Per-screen flow state** (checkout state across multiple Fragments) → @ActivityRetainedScoped
- **Per ViewModel** (rare; usually you'd just constructor-inject) → @ViewModelScoped

> **Common bug:** @Singleton scope on a class that holds an Activity reference → leaks the Activity for the entire app lifetime. Singletons should hold nothing Activity-bound.

## 6. `@Provides` vs `@Binds` — when each

| | @PROVIDES | @BINDS |
|---|---|---|
| Defined in | Object module (or non-abstract class) | Abstract function in abstract class / interface |
| Body | You write the construction logic | Just declares "interface I → impl Class"; no body |
| Use when | Constructing 3rd-party types you don't own (Retrofit, OkHttp) | Binding interface → concrete class you wrote (with @Inject constructor) |
| Bytecode | Generates a factory class that calls your method | Generates one fewer factory class than @Provides — slightly cheaper at build time |

### @Provides — for things you don't construct yourself

```
@Module
@InstallIn(SingletonComponent::class)
object DataModule {
    @Provides @Singleton
    fun provideOkHttp(): OkHttpClient = OkHttpClient.Builder()
        .addInterceptor(LoggingInterceptor())
        .build()
}
```

### @Binds — for "this interface is implemented by this class"

```
interface UserRepository { suspend fun getUser(id: String): User }

class UserRepositoryImpl @Inject constructor(
    private val api: UserApi,
    private val dao: UserDao
) : UserRepository { ... }

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {                // abstract class, not object
    @Binds @Singleton
    abstract fun bindUserRepo(impl: UserRepositoryImpl): UserRepository
}
```

Why use @Binds instead of @Provides for this case?

```
// @Provides equivalent — works but more code + extra factory:
@Provides @Singleton
fun provideUserRepo(api: UserApi, dao: UserDao): UserRepository = UserRepositoryImpl(api, dao)

// @Binds — Hilt sees UserRepositoryImpl already has @Inject constructor;
// just notes "when UserRepository is asked, hand back the UserRepositoryImpl Hilt knows how to make"
```

> **Rule:** use @Binds whenever you have an interface and a concrete impl with @Inject constructor — it's the canonical pattern. Use @Provides for everything else (3rd-party types, things needing custom construction, primitive values, things that need to be lazy).

## 7. Multibinding — `@IntoSet`, `@IntoMap`

Multibinding lets multiple providers contribute to a single Set or Map. Used for plugin systems, interceptor chains, analytics handlers, navigation routes.

### @IntoSet — collect many of the same type

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides @IntoSet
    fun provideAuthInterceptor(tokens: TokenStore): Interceptor =
        AuthInterceptor(tokens)

    @Provides @IntoSet
    fun provideLoggingInterceptor(): Interceptor =
        HttpLoggingInterceptor().apply { level = HttpLoggingInterceptor.Level.BASIC }

    @Provides @Singleton
    fun provideOkHttp(interceptors: Set<@JvmSuppressWildcards Interceptor>): OkHttpClient =
        OkHttpClient.Builder().apply {
            interceptors.forEach { addInterceptor(it) }
        }.build()
}
```

Now any module in any feature can add another `@IntoSet Interceptor` — no need to modify the OkHttp provider. Pure *Open/Closed Principle* in action.

## @IntoMap — keyed selection

```
interface CommandHandler { suspend fun handle(payload: Bundle) }

@MapKey
annotation class CommandKey(val value: String)

@Module
@InstallIn(SingletonComponent::class)
object CommandsModule {
    @Provides @IntoMap @CommandKey("open_url")
    fun provideOpenUrl(impl: OpenUrlHandler): CommandHandler = impl

    @Provides @IntoMap @CommandKey("refresh")
    fun provideRefresh(impl: RefreshHandler): CommandHandler = impl
}

// Consumer just receives the whole map:
class CommandRouter @Inject constructor(
    private val handlers: Map<String, @JvmSuppressWildcards CommandHandler>
) {
    suspend fun dispatch(name: String, payload: Bundle) {
        handlers[name]?.handle(payload)
    }
}
```

> **The `@JvmSuppressWildcards` trick:** Kotlin generics emit Java wildcards ( `Set<? extends Interceptor>` ) which Dagger doesn't recognize. The annotation tells the compiler "use exact types." Required for multibinding consumers.

## 8. Qualifiers — when you have two of the same type

You have two `OkHttpClient`s — one for your auth-required API, one for image loading without auth. How does Hilt know which is which? With qualifiers.

```
@Qualifier @Retention(AnnotationRetention.BINARY)
annotation class Authenticated

@Qualifier @Retention(AnnotationRetention.BINARY)
annotation class Unauthenticated

@Module @InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides @Singleton @Authenticated
    fun provideAuthClient(@Authenticated base: OkHttpClient, auth: AuthInterceptor): OkHttpClient =
        base.newBuilder().addInterceptor(auth).build()

    @Provides @Singleton @Unauthenticated
    fun provideUnauthClient(): OkHttpClient = OkHttpClient()
}

// Consumer specifies which one
class UserRepository @Inject constructor(
    @Authenticated private val client: OkHttpClient
)
```

### Built-in qualifiers worth knowing

| ANNOTATION | WHAT IT GIVES YOU |
|---|---|
| `@ApplicationContext` | Application context (use anywhere) |
| `@ActivityContext` | Activity context (only inside Activity-scoped components) |

```
class CrashLogger @Inject constructor(
    @ApplicationContext private val ctx: Context        // always Application context
)
```

## 9. Constructor injection vs field injection

| | CONSTRUCTOR INJECTION | FIELD INJECTION |
|---|---|---|
| Where | Anywhere YOU control construction (Repos, ViewModels, UseCases) | Where the FRAMEWORK constructs you (Activity, Fragment, Service, BroadcastReceiver, View) |
| Syntax | `@Inject constructor(...)` | `@Inject lateinit var x: X` |
| Testability | Excellent — pass deps as constructor args | Worse — need a way to inject the field in tests |
| Immutability | val (immutable) | lateinit var (mutable, but Kotlin warns if reassigned) |

### Constructor — preferred (90% of your code)

```
class UserRepository @Inject constructor(
    private val api: UserApi,
    private val dao: UserDao
) : UserRepository { ... }

@HiltViewModel
class UserViewModel @Inject constructor(
    private val repo: UserRepository,
    private val handle: SavedStateHandle
) : ViewModel()
```

### Field — required for Android-instantiated components

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    @Inject lateinit var analytics: Analytics       // injected before super.onCreate
    @Inject lateinit var featureFlags: FeatureFlags
}
```

**Trap:** if you use field injection in a class you create yourself ( `val obj = MyClass()` ), the @Inject fields stay **uninitialized** — Hilt only injects through entry points it knows about. For your own classes, always use constructor injection.

## 10. Hilt with ViewModel — `@HiltViewModel`

```
@HiltViewModel
class UserViewModel @Inject constructor(
    private val repo: UserRepository,
    private val handle: SavedStateHandle
) : ViewModel() { ... }

// In Compose
@Composable fun Screen(vm: UserViewModel = hiltViewModel()) { ... }

// In a Fragment
@AndroidEntryPoint
class UserFragment : Fragment() {
    private val vm: UserViewModel by viewModels()
}
```

### Sharing a ViewModel across Composables in a nav graph

```
navigation<CheckoutGraph>(startDestination = CartScreen) {
    composable<CartScreen> { entry ->
        val parent = remember(entry) {
            navController.getBackStackEntry<CheckoutGraph>()
        }
        val sharedVm: CheckoutViewModel = hiltViewModel(parent)
        ...
    }
    composable<PaymentScreen> { entry ->
        val parent = remember(entry) {
            navController.getBackStackEntry<CheckoutGraph>()
        }
        val sharedVm: CheckoutViewModel = hiltViewModel(parent)
        ...
    }
}
```

The shared VM lives until the user pops the entire CheckoutGraph — perfect for multi-step wizards.

### SavedStateHandle

Hilt knows about `SavedStateHandle`; just include it in your ViewModel's constructor and Hilt provides it (with the args from navigation deep links / Activity intent extras pre-populated).

## 11. Hilt with WorkManager

WorkManager creates Worker instances itself, so you can't constructor-inject directly. Use Hilt's `HiltWorkerFactory`.

```
// 1. Worker — annotate
@HiltWorker
class SyncWorker @AssistedInject constructor(
    @Assisted ctx: Context,
    @Assisted params: WorkerParameters,
    private val repo: UserRepository                // regular DI dep
) : CoroutineWorker(ctx, params) {
    override suspend fun doWork(): Result {
        repo.sync()
        return Result.success()
    }
}

// 2. Application — implement Configuration.Provider
@HiltAndroidApp
class App : Application(), Configuration.Provider {
    @Inject lateinit var workerFactory: HiltWorkerFactory

    override val workManagerConfiguration: Configuration
        get() = Configuration.Builder()
            .setWorkerFactory(workerFactory)
            .build()
}

// 3. Manifest — disable WorkManager's default initializer (we provide our own)
<provider
    android:name="androidx.startup.InitializationProvider"
    android:authorities="${applicationId}.androidx-startup"
    android:exported="false"
    tools:node="merge">
    <meta-data android:name="androidx.work.WorkManagerInitializer"
        android:value="androidx.startup"
        tools:node="remove" />
</provider>
```

`@AssistedInject` + `@Assisted` are the dance: WorkManager provides `Context` and `WorkerParameters` (the @Assisted args); Hilt provides everything else.

## 12. Hilt with Compose Navigation

Already shown in §10. The key APIs:

- `hiltViewModel()` — get/create a ViewModel scoped to the current NavBackStackEntry
- `hiltViewModel(parentEntry)` — get/create a ViewModel scoped to a specific parent entry (for shared VMs across a nested graph)

The lifecycle: ViewModel is created when the entry enters the back stack; cleared when the entry is popped (not when it goes off-screen — same as VM survival across rotation).

## 13. `@EntryPoint` — when you can't @Inject

Hilt only injects via `@AndroidEntryPoint`-annotated classes (Activities, Fragments, Services, etc.). For places Hilt doesn't directly support — ContentProviders, classes you don't own, framework callbacks — use `@EntryPoint` to grab dependencies manually.

```
@EntryPoint
@InstallIn(SingletonComponent::class)
interface RepositoryEntryPoint {
    fun userRepository(): UserRepository
}

// Inside a ContentProvider (Hilt doesn't support direct injection here)
class NotesContentProvider : ContentProvider() {
    private val repo: UserRepository by lazy {
        EntryPointAccessors.fromApplication(
            context!!.applicationContext,
            RepositoryEntryPoint::class.java
        ).userRepository()
    }
    ...
}
```

Common reasons:

- ContentProvider (Hilt 2.x doesn't support directly)
- Custom Application init blocks (App Startup library Initializer)
- JobService (Hilt supports Service but not all variants)
- BroadcastReceiver dynamically registered via Context (use @AndroidEntryPoint for manifest-declared receivers)

## 14. ★ What Hilt actually generates — the bytecode walkthrough

Hilt is "Dagger preconfigured for Android." Your KSP/kapt build phase produces three categories of generated classes:

### 1. Component graph — the actual DI engine

For each `@HiltAndroidApp` module, Hilt generates a class like `DaggerApp_HiltComponents_SingletonC` (you can find it in `build/generated/.../hilt`). This class:

- Implements `SingletonComponent`
- Holds singleton instances (lazily constructed)
- Has factory methods for every type Hilt knows how to construct
- Hosts subcomponents — `ActivityRetainedComponent`, `ActivityComponent`, etc. — as nested classes

### 2. `Hilt_*` shells for entry points

For every `@AndroidEntryPoint` class (Activity, Fragment, etc.), Hilt generates a base class that intercepts `onCreate` to inject fields:

```
// You write:
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    @Inject lateinit var analytics: Analytics
}

// Hilt generates (simplified):
class Hilt_MainActivity : ComponentActivity() {
    override fun onCreate(b: Bundle?) {
        inject()                        // resolves dependencies, sets fields
        super.onCreate(b)
    }
    private fun inject() {
        DaggerApp_HiltComponents_SingletonC.create()
            .activityComponent(this)
            .injectMainActivity(this)
    }
}

// Your MainActivity at compile time becomes:
class MainActivity : Hilt_MainActivity() {           // silent rebase
    @Inject lateinit var analytics: Analytics
}
```

Hilt's Gradle plugin does the silent rebase — your code never sees `Hilt_MainActivity` directly.

## 3. Provider factories — one per @Provides / @Binds

For every `@Provides` method, Hilt generates a Factory class:

```
// You write:
@Provides @Singleton
fun provideOkHttp(): OkHttpClient = OkHttpClient.Builder().build()

// Hilt generates:
class NetworkModule_ProvideOkHttpFactory : Provider<OkHttpClient> {
    private val module: NetworkModule
    constructor(module: NetworkModule) { this.module = module }
    override fun get(): OkHttpClient = module.provideOkHttp()

    companion object {
        fun create(module: NetworkModule) = NetworkModule_ProvideOkHttpFactory(module)
    }
}
```

The generated component wires factories together: `Retrofit_Factory` calls into `OkHttp_Factory.get()`; the @Singleton scope wraps each factory in a `DoubleCheck` for thread-safe lazy single-instance behavior.

### 4. Hilt's "aggregating" magic across modules

For multi-module apps, the Hilt processor runs in two phases:

1. **Per-module**: each module emits "metadata" descriptors of its bindings
2. **Aggregating** (in the `:app` module): collects metadata from every dependency, builds the full component graph

This is why adding a new `@Module` in any feature module just works — the app module's aggregating processor picks it up.

### How to inspect what Hilt generated

- Look in `build/generated/ksp/.../hilt/` after a build
- Use Android Studio's "Go to Implementation" on the interface — Hilt-generated impl shows up
- For debugging compile errors, enable `kapt.classloaders.cache.size = 0` + `kapt.include.compile.classpath = false` (slower but clearer error messages)

## 15. Testing with Hilt

### Replace production modules with test modules

```
@TestInstallIn(components = [SingletonComponent::class], replaces = [DataModule::class])
@Module
object FakeDataModule {
    @Provides @Singleton
    fun provideRepo(): UserRepository = FakeUserRepository()
}
```

### Custom Application + custom test runner

```
@CustomTestApplication(BaseApp::class)
interface HiltTestApplication               // generates HiltTestApplication_Application

class HiltTestRunner : AndroidJUnitRunner() {
    override fun newApplication(cl: ClassLoader?, name: String?, ctx: Context?): Application {
        return super.newApplication(cl, HiltTestApplication_Application::class.java.name, ctx)
    }
}
// build.gradle.kts: testInstrumentationRunner = "com.example.HiltTestRunner"
```

### The test itself

```
@HiltAndroidTest
@RunWith(AndroidJUnit4::class)
class NotesScreenTest {
    @get:Rule(order = 0) val hilt = HiltAndroidRule(this)
    @get:Rule(order = 1) val compose = createAndroidComposeRule<HiltTestActivity>()

    @Inject lateinit var repo: UserRepository    // auto-injected = FakeUserRepository

    @Before fun init() { hilt.inject() }

    @Test fun showsNotes() {
        runBlocking { (repo as FakeUserRepository).seed(...) }
        compose.setContent { NotesScreen() }
        compose.onNodeWithText("Hello").assertIsDisplayed()
    }
}
```

**Common pitfalls:** forgetting `hilt.inject()` in @Before → fields are uninitialized; getting the rule order wrong (Hilt rule must be first); forgetting to add the Hilt test runner to Gradle config.

## 16. Hilt + KMP — what works, what doesn't

| SCENARIO | HILT SUITABLE? | RECOMMENDATION |
|---|---|---|
| Android-only app | Yes | Hilt all the way |
| KMP shared module (Android + iOS) | No (Android-only) | **Koin** in commonMain |
| Mixed app: KMP shared module + Android-only ViewModels/UI | Both | Koin in shared, Hilt in Android app — bridge with @Provides that exposes Koin singletons to Hilt graph |

### Bridge pattern for the mixed case

```
// In Android app — Hilt module that pulls from Koin's shared graph
@Module
@InstallIn(SingletonComponent::class)
object SharedBridgeModule {
    @Provides @Singleton
    fun provideUserRepository(): UserRepository =
        SharedKoinGraph.koin.get()        // pulls from KMP shared Koin context
}
```

## 17. Common mistakes

**DON'T**

- @Singleton on a class that holds an Activity (leak)
- @Inject lateinit var in a class you create yourself (won't be injected)
- Apply field injection without @AndroidEntryPoint
- Forget @JvmSuppressWildcards on Set/Map multibinding consumers
- Forget @HiltAndroidApp on Application
- Forget the Hilt test runner in Gradle for instrumented tests
- Put business logic in @Provides methods (move to a helper class)
- Use Hilt for KMP shared code (it's Android-only)
- Override the wrong scope (everything @Singleton becomes a memory leak vector)
- Catch DaggerComponent generation errors at runtime — they're compile-time
- Use @Provides where @Binds suffices (extra factory class)

**DO**

- Default to constructor injection
- Use @Binds for interface → impl wiring
- Use @Provides for 3rd-party constructors
- Pick the smallest scope that satisfies the lifetime
- Inject CoroutineDispatcher (don't hardcode `Dispatchers.IO`)
- Use qualifiers when you have two of the same type
- Use @TestInstallIn modules for tests
- Use @ApplicationContext (never raw Context in singletons)
- Run KSP (faster than kapt) for Hilt processing
- Read the generated component classes when debugging

## 18. Interview Q&A

### Q: "Why use Hilt instead of writing a Service Locator yourself?"

Three reasons. (1) **Compile-time validation** — missing bindings fail the build, not at runtime. (2) **Scope management** — built-in @Singleton, @ActivityRetainedScoped, @ViewModelScoped with proper lifecycle cleanup. (3) **Multi-module aggregation** — modules can declare bindings; the app module aggregates them automatically. A Service Locator at app scale becomes a Gradle-coupled spaghetti graph; Hilt keeps it clean.

### Q: "@Provides vs @Binds — when each?"

@Binds when you have an interface and a concrete impl with @Inject constructor — the canonical pattern. @Provides for everything else: 3rd-party types you can't annotate (OkHttp, Retrofit, etc.), values needing custom construction, primitives.

### Q: "How does Hilt know which scope to give my class?"

You annotate the binding (the @Provides method, @Binds method, or the class itself for @Inject constructor) with the scope annotation: @Singleton, @ActivityRetainedScoped, @ViewModelScoped, etc. Without a scope annotation, the binding is "unscoped" — Hilt creates a fresh instance every time it's requested. This is the default.

### Q: "What's the difference between @Singleton and a Kotlin `object`?"

Both are app-process-singletons. `object` can't have constructor parameters, can't be subbed in tests (no DI). @Singleton-scoped classes have @Inject constructor with dependencies — testable, swappable, future-proof. Use @Singleton for anything with dependencies; use `object` only for stateless utility containers.

### Q: "What is @AssistedInject for?"

For classes whose construction needs both DI dependencies AND runtime parameters. Most common case: WorkManager Workers — they're constructed with `(Context, WorkerParameters)` from WorkManager itself, plus your own deps from Hilt. @AssistedInject + @Assisted lets Hilt provide its part while leaving the rest to be supplied at construction time.

### Q: "How do you share a ViewModel across multiple Composables?"

Scope it to a parent NavBackStackEntry. Use `navController.getBackStackEntry<ParentRoute>()` and pass that to `hiltViewModel(parentEntry)`. The VM lives until the parent destination is popped — perfect for multi-step flows like checkout. Never use Activity-scoped VMs for multi-Fragment flows in 2026 — Compose Nav scoping is the right tool.

### Q: "How do you mock a Repository in a Hilt test?"

Don't mock — fake. Write a `FakeUserRepository` implementing the same interface. In your test source set, create a module annotated `@TestInstallIn(components = [SingletonComponent::class], replaces = [DataModule::class])` that provides the fake. Hilt automatically swaps the production module for your test module in tests.

### Q: "Why is the @JvmSuppressWildcards annotation needed for multibinding?"

Kotlin generics emit Java wildcards by default. `Set<Interceptor>` in Kotlin source becomes `Set<? extends Interceptor>` in bytecode. Dagger looks up bindings by exact type; `Set<? extends Interceptor>` ≠ `Set<Interceptor>` in its lookup table. The annotation tells the compiler "use exact types here." Required for any multibinding consumer.

### Q: "Hilt vs Koin — which would you pick today and why?"

For a pure Android app: **Hilt** — compile-time safety, Android-specific scopes, IDE support. For a KMP project sharing modules between Android and iOS: **Koin** — pure Kotlin, multiplatform. For mixed: Koin in shared module, Hilt in Android app, bridge them via @Provides methods.

### Q: "How does Hilt handle config changes?"

Through scope hierarchy. @Singleton survives everything. @ActivityRetainedScoped survives configuration change (re-uses the ActivityRetainedComponent across destroy/recreate of the Activity). @ActivityScoped is recreated on rotation along with the Activity. ViewModelScoped is per-ViewModel which itself survives rotation via the standard ViewModelStore mechanism.

### Q: "Why does Hilt require KSP/kapt?"

Hilt is annotation-driven. The processor generates the actual DI component graph at compile time — factories per @Provides, the singleton container, entry-point shells. No reflection at runtime; everything's just generated code calling generated factories. KSP is preferred over kapt: 2-3× faster, native Kotlin AST awareness, supported by Hilt 2.48+.

### Q: "How do you debug a Hilt compile error like 'cannot be provided'?"

Read the error carefully — Hilt names the type and which @Inject site asked for it. Common causes: missing @Inject constructor on the class; missing @Module entry; module not @InstallIn the right component; qualifier mismatch (provider has @Auth, consumer doesn't or vice versa); scope mismatch (consumer is @Singleton but dependency is @ActivityScoped — narrower scope can't satisfy wider).

## 19. One-page recall card

### The mental model

DI = receive your dependencies, don't construct them. Hilt = Dagger preconfigured for Android with built-in scopes. Compile-time validation (KSP/kapt). The framework wires constructor-injected classes automatically; you only @Provides what Hilt can't construct (3rd-party types).

### The setup

- @HiltAndroidApp on Application
- @AndroidEntryPoint on Activity / Fragment / Service / View
- @HiltViewModel on ViewModels
- @Module @InstallIn(...) for @Provides / @Binds
- KSP plugin for processing

### Scopes (smallest that fits)

- @Singleton — app process
- @ActivityRetainedScoped — survives rotation, multi-Fragment shared state
- @ViewModelScoped — per VM
- @ActivityScoped / @FragmentScoped / @ViewScoped / @ServiceScoped
- (no annotation) — fresh instance each time

### Bindings

- @Provides — for 3rd-party / custom construction
- @Binds (abstract) — interface → impl with @Inject constructor
- @IntoSet / @IntoMap — multibinding
- @Qualifier — disambiguate same-type dependencies
- @AssistedInject + @Assisted — runtime params (Workers)
- @EntryPoint — manual access from non-Hilt classes

### Testing

- @TestInstallIn replaces production modules
- @HiltAndroidTest + HiltAndroidRule + hilt.inject() in @Before
- Custom test Application + HiltTestRunner in Gradle config

- Prefer fakes over mocks (see Testing cheat sheet)

### Avoid

- @Singleton holding Activity reference (leak)
- Field injection without @AndroidEntryPoint
- Forgetting @JvmSuppressWildcards on Set/Map consumers
- Hilt in KMP shared (use Koin)
- Business logic inside @Provides methods
- Same-type bindings without qualifiers
- kapt when KSP works (faster)

Read top to bottom for understanding · skim §17 (mistakes), §18 (Q&A), §19 (recall) before any DI interview. Pair with the Architecture, Testing, and Coroutines cheat sheets in this repo.